QueryLearn: Defense & Presentation Guide

Simple Student Edition • Everything You Need to Know for Your Defense in Plain Words

1. System Overview: What is QueryLearn? (In Simple Words)

What is it? QueryLearn is an online web platform built for our case study. Its main job is to measure and compare how computer students write and read data queries using two different languages: **SQL** (database queries) vs. **Python** (step-by-step code loops).

Why did we build this? In college (BSCS and BSIT), students learn both SQL and Python. SQL tells the database *what* you want (e.g., SELECT name WHERE score >= 80), while Python requires you to write loops and check conditions line by line. We wanted to see which language is faster, easier to understand, and causes fewer headaches for student beginners.

How a Student Uses the System (Step-by-Step Flow):

Step	What Happens	Why It's There
1. Registration	Student registers with their Program (BSCS/BSIT) and Year Level (2nd, 3rd, 4th). They get a private Study ID (like P001, P002).	Keeps data anonymous and makes sure freshmen are excluded (because SQL starts in 2nd year).
2. Language Guide	Shows a simple reference sheet for SQL commands and Python syntax with clear examples.	Makes sure every student has a fair refresher before they start.
3. Readiness Quiz	A 2-question quick check to confirm they understand what SELECT and for-loops do.	Verifies that the student is ready to take the evaluation.
4. Reading Test (Comprehension)	Students look at 6 short code snippets and predict what the output will be (scored 0 to 18 points).	Measures **code readability** — is SQL or Python easier to read without running it?
5. The 6 Timed Coding Tasks	Students solve 6 data problems with an 8-minute countdown timer per task. An auto-grader checks their output.	Measures **productivity & accuracy** — how fast and correctly students write queries.
6. Student Survey	Students rate their experience (1-5 scale) on ease of use, mental effort, and fatigue.	Captures student feedback and preferences.

2. The 6 Coding Tasks: What Did Students Solve?

Both SQL and Python used identical student database scenarios (Students, Courses, and Enrollments tables). The 6 tasks tested different levels of database skills from simple to complex:

Task ID	What the Student Had to Do	Difficulty / Skill Tested
T1	Find all 2nd-year BSCS students and sort them by student ID.	Basic filtering & sorting (WHERE & ORDER BY)
T2	Find the top 3 highest scores in course C101 (skipping blank scores).	Ordering and picking top rows (LIMIT & NULL check)
T3	Count how many students are in each academic program (BSCS, BSIT, etc.).	Group counting (GROUP BY & COUNT)
T4	Combine 3 tables to show student names and the courses they are taking.	Connecting tables together (INNER JOIN)

T5	Find courses with an average grade of 80 or higher.	Filtering grouped averages (HAVING clause)
T6	Find students who have NEVER enrolled in any course.	Finding missing records (LEFT JOIN ... IS NULL)

3. Study Findings: What Did the Results Prove?

Our study evaluated **28 student participants ($n = 28$)** across 168 total task attempts per language. Here are the core findings and what each number means:

Key Metric	SQL	Python	What This Means in Plain Words
Speed (Time to Finish)	**103.8 seconds**	184.8 seconds	**SQL was ~81 seconds faster.** Writing a SQL query took nearly half the time of writing nested Python loops.
Tasks Done per Hour	**28.2 tasks/hr**	18.4 tasks/hr	**SQL has much higher productivity.** Students can solve roughly 10 more problems every hour using SQL.
Reading Test Score (Comprehension)	**16.0 / 18 pts (78.2%)**	11.0 / 18 pts (63.9%)	**SQL is significantly easier to read and predict.** Students immediately understood SQL clauses without needing mental simulation.
Task Success Rate (Correctness)	83.3% (140/168)	**89.9% (151/168)**	Python had slightly higher raw accuracy (+6.6%). Why? Because students took much longer writing and double-checking loops. The difference was not statistically significant.
T6 (Finding Missing Records)	**100% correct**	80% correct	Writing anti-joins in Python confused students (managing sets vs list of dicts), while SQL's LEFT JOIN was easy.

4. Step-by-Step Defense Script (Word-for-Word Speaking Guide)

Read these sections out loud when presenting to your teacher. Practice them once or twice:

PART 1: GREETING & INTRODUCTION (30 SECONDS)

*"Good morning/afternoon, Sir/Ma'am. Today, I am presenting our case study project called **QueryLearn**.*

*QueryLearn is an online evaluation platform that compares how student developers read and write database queries using two different approaches: **SQL** versus **procedural Python**.*

As computing students, we are taught both languages for data handling. Our goal with QueryLearn was to conduct an objective case study to find out which language is faster, easier to understand, and more efficient for students solving everyday data retrieval tasks."

PART 2: SYSTEM ARCHITECTURE & DEMO (1 MINUTE)

"Let me give you a quick walkthrough of how the system works:

*1. **Registration:** Students sign up with their year level and course. First-year students are excluded because database instruction starts in 2nd year.*
*2. **Fair Order (Counterbalancing):** To be fair, half of the students do SQL first, and the other half do Python first. This prevents any order bias.*
*3. **Reading Comprehension Test:** Before coding, students predict the output of 6 code snippets. This measures code readability.*
*4. **Coding Tasks:** In the workspace, students write queries for 6 standard tasks under an 8-minute timer. Our automated grading engine checks their output against hidden datasets in real time.*
*5. **Cloud Database & Dashboard:** All attempts, execution times, and survey answers are stored in a cloud database (Supabase) and synced so the researcher dashboard can display live analytics."*

PART 3: EXPLAINING THE FINDINGS & CHARTS (1.5 MINUTES)

"Now, looking at our live dashboard and results from the 28 participants:

*• **First, on Writing Speed:** SQL was clearly faster. The average time to reach a correct solution in SQL was **103.8 seconds**, compared to **184.8 seconds** in Python. That means SQL was about 81 seconds faster per task, giving SQL a **1.82x speed advantage**.*

*• **Second, on Productivity:** Students completed **28.2 tasks per hour in SQL** versus only **18.4 tasks per hour in Python**. SQL allowed students to get nearly 10 more tasks done per hour.*

*• **Third, on Readability:** In the comprehension test, SQL had an average score of **16.0 out of 18** (78.2%), while Python had **11.0 out of 18** (63.9%). Students found SQL significantly easier to read and predict without running the code.*

*• **Fourth, on Accuracy:** Python had a slightly higher raw success rate (89.9% vs 83.3%), but this was because students spent much more time carefully debugging their loops. However, on complex operations like Task 6 (anti-joins), all students got SQL right (100%), while only 80% succeeded in Python."*

PART 4: CONCLUSION & RECOMMENDATION (30 SECONDS)

*"In conclusion, our case study proves that for relational data retrieval, **SQL offers superior productivity, faster completion times, and higher code readability** for student learners. While Python is powerful for general programming, having to write manual loops and dictionary lookups slows down query formulation.*

We recommend that educational curricula introduce SQL early for database querying, while positioning Python as an analytical tool that processes data after it has been retrieved via SQL.

Thank you, Sir/Ma'am! I am now ready to answer any questions you may have."

5. Anticipated Teacher Questions & Exact Answers (Q&A; Cheat Sheet)

Your teacher might ask these common questions during your defense. Here is exactly what to say:

Q1: Why did you compare SQL and Python? Aren't they used for different purposes?

What to Answer: *"Sir/Ma'am, in modern data science and software development, developers often manipulate tabular data using both SQL queries and Python scripts (like list/dict operations or Pandas). We wanted to see whether students grasp declarative queries (SQL) faster than procedural logic (Python loops) when handling the exact same dataset."*

Q2: Why did Python have an 89.9% accuracy rate while SQL had 83.3%?

What to Answer: *"That is a great observation, Sir/Ma'am. Students spent much longer in Python (averaging 184 seconds compared to 103 seconds in SQL). Because they spent more time writing and testing loops, slightly more students eventually solved the tasks. However, this difference of 6.6% was not statistically significant ($p = 0.366$), whereas SQL was statistically significantly faster in speed and tasks per hour."*

Q3: Why are freshmen (1st Year) excluded from the study?

What to Answer: *"In our college curriculum, formal database management and SQL instruction begin in the second year. First-year students have not yet taken a database course, so including them would skew the results toward zero SQL familiarity."*

Q4: How does the system grade students automatically?

What to Answer: *"When a student clicks 'Submit', the system runs their code in a sandboxed runner and compares their table output against expected test datasets. If their column names, rows, and values match the hidden test key, it automatically marks it correct; otherwise, it records the error message and lets them retry until the 8-minute timer expires."*

Q5: What technology stack does QueryLearn use?

What to Answer: *"The frontend uses HTML, CSS, JavaScript, and Chart.js for interactive charts. The backend is built with Python Flask and hosted on Vercel. For database storage, we use SQLite for local execution and Supabase (PostgreSQL) in the cloud to sync participant results across different computers in real time."*

Q6: Why did you use counterbalancing (Sequence 1 to 4)?

What to Answer: *"If all students did SQL first and then Python, they might do better in Python simply because they already saw the problem in SQL (called a 'learning effect'). By having half the students start with SQL and the other half start with Python, the study remains fair and unbiased."*